

Experiment 01:

Laboratory Exercise

❖ Numpy

1. Using np.array

Code:

```
import numpy as np

arr=[1,2,3,4,5]

#1D array

arr1D=np.array(arr)
print("-----1D Array-----")
print(arr1D)

#2D array : (rows,cols)

arr2=[[1,2,3],[4,5,6],[7,8,9]]
arr2D=np.array(arr2)
print("-----2D Array-----")
print(arr2D)

#3D array : (depth, rows, cols)

arr3=[[[1,2],[3,4]],[[5,6],[7,8]]]
arr3D=np.array(arr3)
print("-----3D Array-----")
print(arr3D)
```

Output:

```
... -----1D Array-----
[1 2 3 4 5]
-----2D Array-----
[[1 2 3]
 [4 5 6]
 [7 8 9]]
-----3D Array-----
[[[1 2]
   [3 4]]
 [[5 6]
   [7 8]]]
```

2. Using Numpy Functions

Code:

```
print("-----1D Array-----")
print("Array :", arr1D)
print("Type :", type(arr1D))
print("Dimensions :", arr1D.ndim)
print("Shape :", arr1D.shape)
print("Size :", arr1D.size)
print("Datatype :", arr1D.dtype)

print("\n-----2D Array-----")
shape2 = arr2D.shape
rows2 = shape2[0]
cols2 = shape2[1]

print("Number of rows :", rows2)
print("Number of cols :", cols2)
print("Array :")
print(arr2D)
print("Type :", type(arr2D))
print("Dimensions :", arr2D.ndim)
print("Shape :", arr2D.shape)
print("Size :", arr2D.size)
print("Datatype :", arr2D.dtype)

print("\n-----3D Array-----")
shape3 = arr3D.shape
depth3 = shape3[0]
rows3 = shape3[1]
cols3 = shape3[2]

print("Number of depths :", depth3)
print("Number of rows :", rows3)
print("Number of cols :", cols3)
print("Array :")
print(arr3D)
print("Type :", type(arr3D))
print("Dimensions :", arr3D.ndim)
print("Shape :", arr3D.shape)
print("Size :", arr3D.size)
print("Datatype :", arr3D.dtype)
```

Output:

```
... -----1D Array-----
Array : [1 2 3 4 5]
Type : <class 'numpy.ndarray'>
Dimensions : 1
Shape : (5,)
Size : 5
Datatype : int64

-----2D Array-----
Number of rows : 3
Number of cols : 3
Array :
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Type : <class 'numpy.ndarray'>
Dimensions : 2
Shape : (3, 3)
Size : 9
Datatype : int64

-----3D Array-----
Number of depths : 2
Number of rows : 2
Number of cols : 2
Array :
[[[1 2]
  [3 4]]
 [[5 6]
  [7 8]]]
Type : <class 'numpy.ndarray'>
Dimensions : 3
Shape : (2, 2, 2)
Size : 8
Datatype : int64
```

3. NumPy Array Indexing

Code:

```
print("-----Array Indexing-----")
print(arr2D[0,0])
print(arr2D[1,2])
print(arr3D[0,1,1])
print(arr3D[1,0,0])
```

Output:

```
... -----Array Indexing-----
1
6
4
5
```

4. Slicing

Code:

```
print("-----Slicing-----")

print(arr1D[0:5])
print(arr1D[1:4])
print(arr1D[:3])
print(arr1D[2:])
print(arr1D[::-1])

print(arr2D[0:3])

print(arr3D[1:2])
```

Output:

```
... -----Slicing-----
[1 2 3 4 5]
[2 3 4]
[1 2 3]
[3 4 5]
[5 4 3 2 1]
[[1 2 3]
 [4 5 6]
 [7 8 9]]
[[[5 6]
  [7 8]]]
```

5. Advanced Indexing

Code:

```
print("-----Advanced Indexing-----")

print(arr1D[[0,2,4]])

print(arr1D[arr1D>3])
```

Output:

```
... -----Advanced Indexing-----
[1 3 5]
[4 5]
```

6. Basic Arithmetic Operations

Code:

```
print("-----Basic Arithmetic Operations-----")

print(arr1D+5)
print(arr1D-2)
print(arr1D*2)
print(arr1D/2)
```

Output:

```
... -----Basic Arithmetic Operations-----
[ 6  7  8  9 10]
[-1  0  1  2  3]
[ 2  4  6  8 10]
[0.5 1.  1.5 2.  2.5]
```

7. Binary Operators

Code:

```
print("-----Binary Operators-----")

print(arr1D>2)
print(arr1D<4)
print(arr1D==3)
print(arr1D!=5)
```

Output:

```
... -----Binary Operators-----
[False False  True  True  True]
[ True  True  True False False]
[False False  True False False]
[ True  True  True  True False]
```

8. Mathematical Functions

Code:

```
print("-----Mathematical Functions-----")

print(f"addition           : {np.sum(arr2D)}")
print(f"average            : {np.mean(arr2D)}")
print(f"minimum             : {np.min(arr2D)}")
```

```
print(f"maximum          : {np.max(arr2D)}")
print(f"standard deviation : {np.std(arr2D)}")
print(f"variance          : {np.var(arr2D)}")
```

Output:

```
... -----Mathematical Functions-----
addition          : 45
average           : 5.0
minimum           : 1
maximum           : 9
standard deviation : 2.581988897471611
variance           : 6.666666666666667
```

9. Sorting Arrays

Code:

```
arr=[9,2,5,8,1,7,4]

arr1D=np.array(arr)

print("-----Sorting Arrays-----")

print(f"Original Array : {arr1D}")
print(f"Sorted Array    : {np.sort(arr1D)}")
```

Output:

```
... -----Sorting Arrays-----
Original Array : [9 2 5 8 1 7 4]
Sorted Array   : [1 2 4 5 7 8 9]
```

❖ Pandas

1. Pandas Data Frames

Code:

```
import pandas as pd

dict1 = {
    "calories": [1000, 2000, 3000],
    "duration (in min)": [10, 60, 120]
}

df = pd.DataFrame(dict1)
print(df)
```

Output:



	calories	duration (in min)
0	1000	10
1	2000	60
2	3000	120

2. Pandas Series

Code:

```
series = pd.Series([10, 20, 30, 40])
print(series)
```

Output:



```
0    10
1    20
2    30
3    40
dtype: int64
```

3. Operations in Pandas- (Loading and viewing data)

Code:

```
iris = pd.read_csv("/kaggle/input/iris-dataset/iris.csv")
df1 = pd.DataFrame(iris)

print(df1)
```

```

print("\n-----HEAD-----\n")
print(df1.head)
print("\n-----TAIL-----\n")
print(df1.tail)
print("\n-----INFO-----\n")
print(df1.info)
print("\n-----SHAPE-----\n")

```

Output:

```

      sepal_length  sepal_width  petal_length  petal_width  species
0              5.1           3.5           1.4           0.2   setosa
...            1
2              4.7           3.2           1.3           0.2   setosa
3              4.6           3.1           1.5           0.2   setosa
4              5.0           3.6           1.4           0.2   setosa
..            ..
145            6.7           3.0           5.2           2.3  virginica
146            6.3           2.5           5.0           1.9  virginica
147            6.5           3.0           5.2           2.0  virginica
148            6.2           3.4           5.4           2.3  virginica
149            5.9           3.0           5.1           1.8  virginica

```

[150 rows x 5 columns]

-----HEAD-----

```

      sepal_length  sepal_width  petal_length  petal_width  species
0              5.1           3.5           1.4           0.2   setosa
1              4.9           3.0           1.4           0.2   setosa
2              4.7           3.2           1.3           0.2   setosa
3              4.6           3.1           1.5           0.2   setosa
4              5.0           3.6           1.4           0.2   setosa

```

-----TAIL-----

```

      sepal_length  sepal_width  petal_length  petal_width  species
145            6.7           3.0           5.2           2.3  virginica
146            6.3           2.5           5.0           1.9  virginica
147            6.5           3.0           5.2           2.0  virginica
148            6.2           3.4           5.4           2.3  virginica
149            5.9           3.0           5.1           1.8  virginica

```

```
-----INFO-----  
  
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 150 entries, 0 to 149  
Data columns (total 5 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   sepal_length    150 non-null    float64  
1   sepal_width     150 non-null    float64  
2   petal_length    150 non-null    float64  
3   petal_width     150 non-null    float64  
4   species         150 non-null    object  
dtypes: float64(4), object(1)  
memory usage: 6.0+ KB  
None  
  
-----SHAPE-----  
  
(150, 5)
```

4. Create Pandas DataFrame from a CSV file

Code:

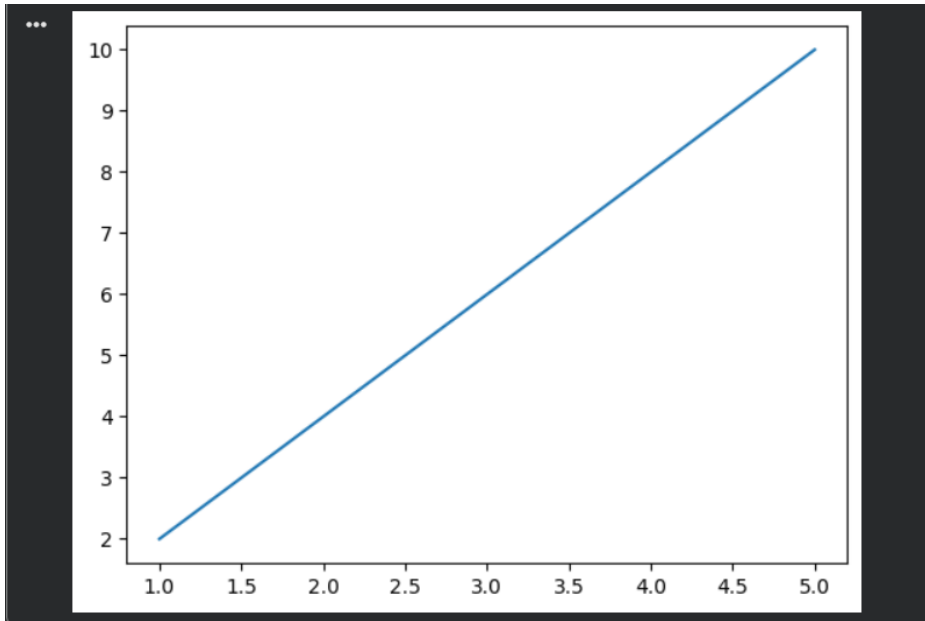
```
iris=pd.read_csv("/kaggle/input/iris-dataset/iris.csv")  
df1 = pd.DataFrame(iris)
```

❖ Matplotlib

1. Plotting a simple linear graph

Code:

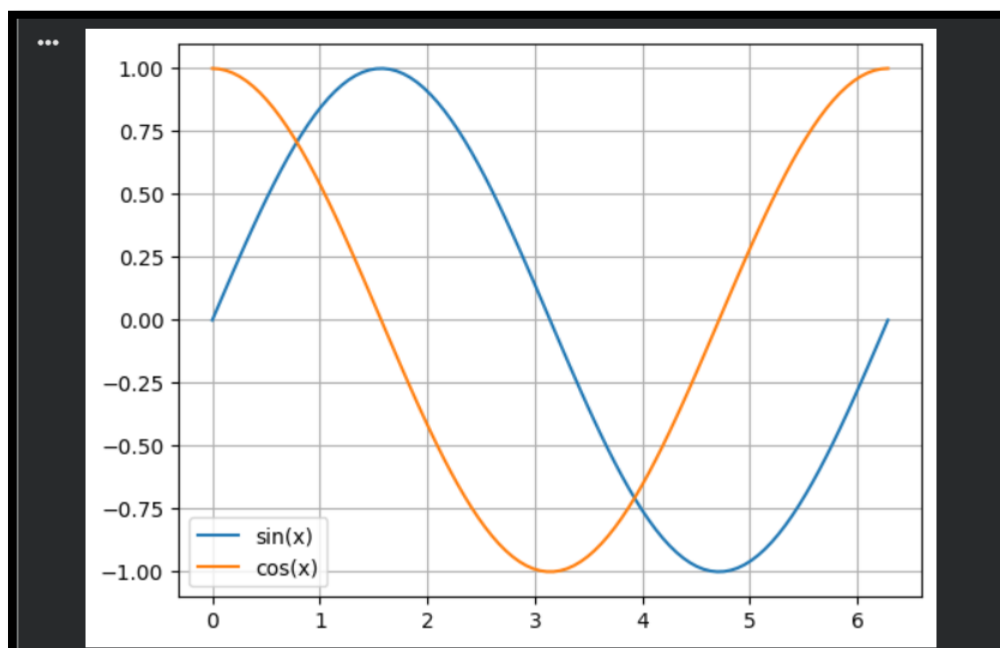
```
import matplotlib.pyplot as plt  
  
X = [1,2,3,4,5]  
Y = [2,4,6,8,10]  
  
plt.plot(X,Y)  
plt.show()
```

Output:**2. Plotting trigonometric functions****Code:**

```
x = np.linspace(0, 2*np.pi, 100)

plt.plot(x, np.sin(x), label="sin(x)")
plt.plot(x, np.cos(x), label="cos(x)")

plt.legend()
plt.grid(True)
plt.show()
```

Output:

Post Experimental Exercise:

1. Difference between Pandas Series and DataFrame? Explain with examples (two examples)

Pandas Series	Pandas DataFrame
A Series is a one-dimensional labeled array.	A DataFrame is a two-dimensional labeled table (like a spreadsheet or SQL table).
It contains a single column of data.	It contains multiple rows and columns.
It stores one-dimensional data.	It stores two-dimensional tabular data.
It is generally homogeneous (stores one type of data).	Different columns can store different data types.
It supports vectorized operations for fast computation.	It also supports vectorized operations on rows and columns.
It can handle missing values using NaN.	It can also handle missing values using NaN.
Both values and structure can be modified.	Both values and structure (rows/columns) can be modified.
It has an index for labeling data.	It has both row indexes and column labels.
It is similar to a single column in a spreadsheet.	It is similar to an entire spreadsheet or SQL table.
Created using pd.Series().	Created using pd.DataFrame().
A Series cannot contain multiple columns.	Each column in a DataFrame is a Pandas Series.

Example 1:

Code (Series):

```
import pandas as pd
marks = pd.Series([85, 90, 63])
print(marks)
```

Output:

```
0    85
1    90
2    78
dtype: int64
```

Code (DataFrame):

```
import pandas as pd

student = {
    "Name": ["Shivam", "Radha", "Ashish"],
    "Marks": [85, 90, 63],
    "Grade": ["A+", "O", "B+"]
}

df = pd.DataFrame(student)
print(df)
```

Output:

```
...      Name  Marks  Grade
0  Shivam     85      A+
1  Radha     90      A+
2  Ashish     63      B+
```

Example 2:**Code (Series):**

```
print("-----Custom Indexing-----")
s2 = pd.Series([36, 37, 38, 39], index = ["Ashish", "Shivam", "Radha",
"Tanish"])
print(s2["Shivam"])
print(s2["Radha"])

print("\n-----Vectorized Operations-----\n")
s1 = pd.Series([1, 2, 3])
s2 = pd.Series([4, 5, 6])
print(s1 + s2)

print("\n-----Mutable Values but immutable size-----\n")
s = pd.Series([1, 2, 3, 4, 5])
s[0] = 100
print(s)
changed_s = s.drop(1)
print(changed_s)
print(s)
```

Output:

```

... -----Custom Indexing-----
37
38

-----Vectorized Operations-----

0    5
1    7
2    9
dtype: int64

-----Mutable Values but immutable size-----

0    100
1     2
2     3
3     4
4     5
dtype: int64
0    100
2     3
3     4
4     5
dtype: int64
0    100
1     2
2     3
3     4
4     5
dtype: int64

```

Code (DataFrame):

```

print("-----Row and Column Labels-----")
info = {
    "Name": ["Ashish", "Shivam", "Radha", "Tanish"],
    "Marks": [36, 37, 38, 39]
}

df = pd.DataFrame(info)

print("Row Labels :")
print(df.index)

print("\nColumn Labels :")
print(df.columns)

```

```
print("-----Vectorized Operations-----")
df["Marks"] = df["Marks"] + 5

print(df)

print("-----Mutable Values-----")
df.at[0, "Marks"] = 100

print(df)

print("-----Each Column is a Series-----")
print(type(df["Name"]))
print(df["Name"])
```

Output:

```
... -----Row and Column Labels-----
Row Labels :
RangeIndex(start=0, stop=4, step=1)

Column Labels :
Index(['Name', 'Marks'], dtype='object')
-----Vectorized Operations-----
   Name  Marks
0  Ashish    41
1  Shivam    42
2   Radha    43
3  Tanish    44
-----Mutable Values-----
   Name  Marks
0  Ashish   100
1  Shivam    42
2   Radha    43
3  Tanish    44
-----Each Column is a Series-----
<class 'pandas.core.series.Series'>
0    Ashish
1    Shivam
2     Radha
3    Tanish
Name: Name, dtype: object
```

2. Create a NumPy array of 10 elements and perform the following:

- Find mean, median, and standard deviation
- Reshape the array into 2×5

Code:

```
import numpy as np
original_array = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

print("-" * 40)
print("Array Statistics")
print("-" * 40)
print(f"Mean           : {np.mean(original_array)}")
print(f"Median          : {np.median(original_array)}")
print(f"Standard Deviation : {np.std(original_array)}")

reshaped_array = original_array.reshape(2, 5)

print("\n" + "-" * 40)
print("Array Reshaping")
print("-" * 40)
print(f"Original Shape      : {original_array.shape}")
print(f"Reshaped Shape      : {reshaped_array.shape}")

print("\nOriginal Array:")
print(original_array)

print("\nReshaped Array:")
print(reshaped_array)
```

Output:

```
... -----
Array Statistics
-----
Mean           : 5.5
Median          : 5.5
Standard Deviation : 2.8722813232690143

-----
Array Reshaping
-----
Original Shape      : (10,)
Reshaped Shape      : (2, 5)

Original Array:
[ 1  2  3  4  5  6  7  8  9 10]

Reshaped Array:
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]]
```

3. Write a Python program to:

- Create a Pandas DataFrame with student details (Name, Marks, Grade)
- Display only students scoring more than 80 marks

Code:

```
import pandas as pd
student_data = {
    "Name": ["Shivam", "Radha", "Ashish", "Aryan", "Abhijeet"],
    "Marks": [99, 100, 77, 85, 86],
    "Grade": ["O", "O", "A", "A+", "A+"]
}

student_dataframe = pd.DataFrame(student_data)
print("Complete DataFrame")
print(student_dataframe)
print("\nStudents Scoring More Than 80 Marks")
print(student_dataframe[student_dataframe["Marks"] > 80])
```

Output:

```
... Complete DataFrame

Students Scoring More Than 80 Marks
   Name  Marks Grade
0  Shivam    99     O
1  Radha   100     O
3  Aryan    85    A+
4 Abhijeet    86    A+
```

4. Using Matplotlib:

- Plot a line graph for given data:
X = [1,2,3,4,5], Y = [2,4,6,8,10]
- Add title, labels, and grid

Code:

```
import matplotlib.pyplot as plt

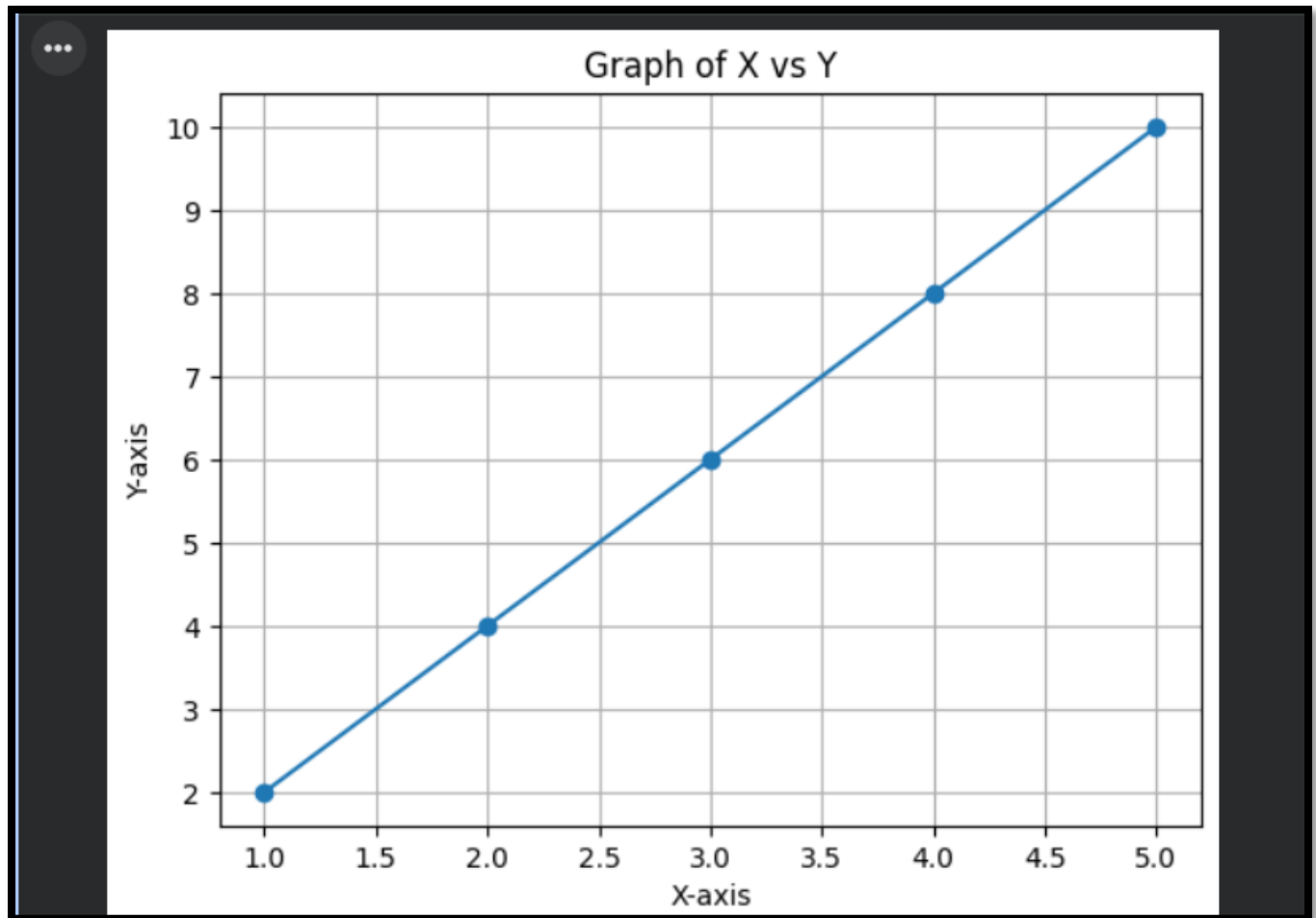
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.plot(x, y, marker="o")

plt.title("Graph of X vs Y")
```

```
plt.xlabel("X-axis")  
plt.ylabel("Y-axis")
```

```
plt.grid(True)  
plt.show()
```

Output:

5. Load the Iris dataset using Scikit-learn and:

- Display first 5 records
- Print feature names and target classes

Code:

```
from sklearn.datasets import load_iris  
  
iris = load_iris()  
  
iris_dataframe = pd.DataFrame(  
    iris.data,  
    columns=iris.feature_names
```

```
)

print("Available keys in iris dataset")
print(iris.keys(),"\n")

print("First 5 Records")
print(iris_dataframe.head())

print("\nFeature Names")
print(iris.feature_names)

print("\nTarget Classes")
print(iris.target_names)
```

Output:

```
Available keys in iris dataset
dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])

First 5 Records
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2
3                4.6                3.1                1.5                0.2
4                5.0                3.6                1.4                0.2

Feature Names
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Target Classes
['setosa' 'versicolor' 'virginica']
```